

План комплексного улучшения ДавидСклад

Magic Vibes - Склад: анализ кодовой базы, архитектуры и дорожная карта развития
Дата: 8 июня 2026 | Режим: только чтение (без изменений кода)

1. Резюме для руководства

ДавидСклад - монолитное Node.js/Express-приложение (~30 000 строк server.js) с двумя UI: современный React SPA (/app/*) и legacy HTML/JS. Ядро: каталог товаров Ozon/Yandex, привязки к PriceMaster (MySQL), автоматизация цен/остатков, автокорзина заказов, лист сборки, финансы и AI-контент. PostgreSQL + Redis/BullMQ внедряются поэтапно.

- Сильные стороны: зрелая бизнес-логика ценообразования, автокорзина с записью в PriceMaster, операции через BullMQ, аудит действий, развитый modern UI для склада и сборки.
- Главные пробелы: нет real-time (только polling), фрагментированный UX (12+ разделов, часть скрыта от manager), журнал аудита без единой ленты «последних действий», дублирование legacy/modern интерфейсов.
- Ginger: в коде нет упоминаний «Ginger». Автокорзина пишет заказы в MySQL PriceMaster (таблицы RequestDocs/RequestRows). Внешняя программа заказов (вероятно Ginger) читает эти строки.
- Приоритет: единый дашборд статусов, улучшение автокорзины (видимость, ошибки, retry), консолидация UI, SSE/WebSocket для маркетплейсов и очередей.

2. Текущая архитектура

2.1. Компоненты системы

```
[Браузер]
|-- /app/* --> React 19 + Vite + TanStack Query (modern UI)
|-- /product.html, /app.js, /pricemaster.html --> Legacy UI
v
[Express 5 server.js + routes/*]
|-- Auth (сессии, роли admin/manager)
|-- API /api/* (~100+ endpoints)
|-- BullMQ Worker (marketplace jobs)
v
[Хранилища]
PostgreSQL (Prisma) - warehouse, links, audit, supplier_cart, picking, finance
JSON fallback (data/*.json) - переходный режим
MySQL PriceMaster - OfferRows, RequestDocs, RequestRows (чтение + запись корзины)
Redis - очереди BullMQ
v
[Внешние API]
Ozon Seller API | Yandex Market API | OpenAI (AI-фото/контент)
```

2.2. Поток данных ценообразования

```
PriceMaster OfferRows (MySQL)
--> snapshot pm_snapshot_items (PostgreSQL) + live-запросы
--> product_links (привязки артикулов к warehouse_products)
--> формула: USD x курс x коэффициент маркетплейса x наценка SKU
```

--> sales_automation_sku_states --> push цен/остатков Ozon/Yandex
--> price_retry_queue при ошибках

2.3. Роли и доступ

Роль	Доступ в modern UI	Ограничения
admin	Каталог, Сборка, Поставщики, Автокорзина, Настройки	Полный доступ к скрытым разделам
manager	Каталог, Сборка	Операции, финансы, автокорзина - запрещены
system	Фоновые задачи	scheduler, daily sync, auto cart

3. Инвентаризация модулей

Модуль	Функции	UI
Каталог (Warehouse)	Фильтры, группировка Ozon/Yandex, привязки PM, диагностика и ремонт, repair	ModernCatalogPage
PriceMaster	Поиск строк, snapshot, live matches, валюта USD/RUB, trust/reason	APISettings
Поставщики	Managed suppliers, стоп-лист, артикулы, ledger	ModernSuppliersPage
Автокорзина	Preview Ozon/Yandex orders, scoring, commit в PM, scheduler	SupplierCartPage
Лист сборки	open/picked/missing, блокировка поставщика, finance/ledger	PickingListPage
Sales automation	Авто цены/остатки, SKU states, verification pending	PricesPage, SystemPage
Ozon unarchive	Очередь разархивации, daily limit, auto-process	RecoveryQueuePage
Операции	BullMQ jobs: import, recovery, AI drafts, repair, health	OperationsPage
Финансы	finance_orders, expenses, profit по заказам	FinancePage
Проблемные товары	Диагностика и repair SKU	ProblemProductsPage
AI drafts	Yandex quality, изображения, контент	AiDraftsPage
Настройки	Курс, наценки, AI, branding, аудит, sync	SettingsPage
Система	Health, postgres, redis, slow endpoints	SystemPage
Legacy	product.html, ozon/yandex export, pricemaster.html	public/*

3.1. Фоновые процессы (server.js)

- Daily sync (DAILY_SYNC_TIME) - PM snapshot + warehouse sync + optional price push
- Auto sync (AUTO_SYNC_MINUTES) - периодическая синхронизация склада
- Marketplace maintenance - фоновое обслуживание маркетплейсов
- Supplier cart auto - 3 раза в день MSK: generate + commit + picking rows
- Ozon unarchive queue auto - обработка очереди разархивации
- Background price push - delta PM changes + marketplace changes

4. Ginger / Автокорзина - как работает сейчас

4.1. Важное уточнение про Ginger

В репозитории нет строк «Ginger» или «ginger». Автокорзина интегрирована с PriceMaster (MySQL, PM_DB_*). Внешняя программа заказов поставщиков (которую вы называете Ginger) скорее всего работает с теми же таблицами RequestDocs/RequestRows в базе PriceMasterDB. ДавидСклад создает документы заказа; Ginger/PriceMaster-клиент их обрабатывает.

4.2. Таблицы PostgreSQL (Prisma)

Таблица	Назначение	Ключевые поля
supplier_cart_drafts	Активный черновик корзины	generatedAt, summary, active, params
supplier_cart_draft_rows	Строки черновика	cartKey, offerId, partnerId, offerRowId, ready, requestDocId
supplier_picking_rows	Лист сборки после commit	pickingKey, status, requestDocId, supplierScore
supplier_blocks	Блок поставщика при «не было»	offerId+partnerId, expiresAt 7 дней
supplier_ledger_entries	Долг/оплата поставщикам	pickingKey, financeOrderId

4.3. Таблицы PriceMaster MySQL (запись автокорзины)

Таблица	Операция	Поля
RequestDocs	INSERT на партнера	DocID, DocDate, PartnerID, Comment='ДавидСклад автокорзина', Registered
RequestRows	INSERT строки заказа	RowID, OfferRowID, RequestQuant, RequestComment, DocID
OfferRows	READ при resolve	RowID, NativeID, NativePrice, Active - выбор поставщика

4.4. Пошаговый поток автокорзины

1. SCHEDULER (09:30, 12:00, 15:00 MSK) или ручной POST /api/supplier-cart/generate
2. fetchOzonSupplierCartLines - postings awaiting_packaging (lookback 48h)
fetchYandexSupplierCartLines - PROCESSING/STARTED
3. resolveSupplierCartRow для каждой строки:
 - a) найти warehouse product по offerId
 - b) взять product_links группы
 - c) getLivePriceMasterMatchesForLinks
 - d) исключить supplier_blocks и cutoff passed
 - e) supplierCartOrderScore: цена + trust + reseller + cutoff
4. generateSupplierCartDraft -> supplier_cart_drafts + rows (PostgreSQL)
5. insertSupplierCartRowsIntoPriceMaster (auto или POST /commit):
 - a) GET_LOCK('davidsklad_supplier_cart')
 - b) группировка по PartnerID -> RequestDocs
 - c) RequestRows с OfferRowID выбранного поставщика
 - d) verifyPriceMasterInsertedRows
 - e) state.processed[cartKey] = requestDocId/requestRowId
6. createSupplierPickingRows -> supplier_picking_rows (status=open)
7. Сотрудник в PickingList: picked/missing -> finance + ledger + blocks

4.5. Состояние JSON (fallback)

data/supplier-cart-state.json: draft, processed{}, history[], supplierBlocks{}. При DB_MODE=postgres приоритет у PostgreSQL с JSON fallback.

4.6. Предложения по улучшению автокорзины/Ginger

- Панель связи PM<->Ginger: показывать статус Sended/Recieved из RequestDocs в real-time
- Webhook или polling Ginger API (если есть) для подтверждения принятия заказа
- Уведомления при skipReason: no_pricemaster_link, supplier_blocked, cutoff_passed
- Dry-run режим с симуляцией без INSERT в PM для тестирования scoring
- Dashboard «строки в PM без picking» и «picking без PM row»
- Автоматический retry при 409 PriceMaster cart is busy
- Документация маппинга: cartKey -> RequestRowID -> Ginger order ID
- Режим partial commit с явным UI выбора (уже есть keys[], улучшить UX)

5. Дорожная карта UI/UX

5.1. Видение дизайна

Единый рабочий стол «операционный день»: утро - автокорзина и сборка, день - каталог и цены, вечер - финансы и аудит. Визуальный язык: тёмно-синяя палитра (#16213e), акцентные статусы (success/warn/danger), карточки вместо таблиц на мобильных, sticky-навигация по ролям.

5.2. Проблемы текущего UI

- Admin видит 5 пунктов меню, остальные 7 разделов недоступны из nav (operations, prices, finance...)
- Legacy /product.html и modern /app/warehouse дублируют функции
- Нет глобального поиска SKU/заказа
- Аудит скрыт в Settings, нет виджета «последние действия»
- Supplier cart и Operations разделены, хотя связаны
- Нет dark mode, нет адаптивной нижней навигации для склада

5.3. Этапы редизайна

Этап	Срок	Результат
UX-0	1-2 нед	Единое меню по ролям, breadcrumbs, global search stub
UX-1	3-4 нед	Dashboard home: KPI + последние действия + очереди
UX-2	1-2 мес	Консолидация legacy в modern, единый product drawer
UX-3	2-3 мес	Design system (tokens), мобильный picking-first UI

6. Real-time статусы маркетплейсов

6.1. Текущее состояние (polling)

Компонент	Интервал	Endpoint
SystemPage	30 сек	/api/system/status
SupplierCart schedule	30 сек	/api/supplier-cart/schedule
PickingList	15 сек	/api/supplier-picking-list
Operations detail	3 сек (active)	/api/operations/:id
Live status	по запросу	/api/live-status

WebSocket/SSE в проекте отсутствуют.

6.2. Целевая архитектура real-time

[Event sources]
BullMQ job progress | sales_automation updates | ozon queue | supplier cart runs
marketplace API webhooks (если доступны)
v
[SSE /api/events или WebSocket]
channel: system | cart | picking | prices | operations/{id}
v
[React Query invalidate / optimistic UI]

6.3. Приоритетные статусы для real-time

- Прогресс операций (BullMQ) без polling 3 сек
- Автокорзина: running -> result с docIds
- Sales automation: price_status/stock_status per SKU batch
- Ozon unarchive queue: due count, verification pending
- PM snapshot refresh: items/changes count
- Marketplace order status для строк автокорзины

7. Журнал / последние действия / аудит

appendAudit() пишет в audit_logs (PostgreSQL) + audit-log.jsonl. API: GET /api/audit-log (admin), GET /api/warehouse/products/audit (link changes per product). UI: SettingsPage -> AuditSettingsPanel (120 записей, поиск). Legacy: public/settings.js. Employee stats в Settings (links created/updated).

7.1. Типы действий (выборка)

- warehouse.link.save, warehouse.links.bulk_save, warehouse.link.delete
- supplier_cart.draft_generate, supplier_cart.commit, supplier_cart.supplier_blocked
- warehouse.product.update, warehouse.markups.bulk_update, sales-automation
- ozon.unarchive_queue.*, supplier_ledger.payment_create, warehouse.ai_*

7.2. Улучшения журнала

- Глобальная лента «Последние действия» на главном дашборде (все роли по фильтру)
- Фильтры: дата, пользователь, entityType, marketplace, offerId
- Diff-view oldValue/newValue для привязок и цен

- Экспорт CSV/JSON, retention policy
- Корреляция audit <-> operation job id
- Push-уведомление admin при supplier_cart.commit ошибке

8. Операции маркетплейсов (Ozon/Yandex)

8.1. Текущие возможности

- Импорт/синхронизация товаров, группировка Ozon<->Yandex pairs
- Push цен: POST /v1/product/import/prices (Ozon), offer-prices/updates (Yandex)
- Push остатков, unarchive queue с daily limit
- Yandex quality AI drafts, repair weak ozon, problem products
- Ozon product create (/ozon-product.html), Yandex export
- Sales automation: расчет targetPrice/targetStock из PM availability rules

8.2. Пробелы и улучшения

Область	Пробел	Улучшение
Верификация	verificationPending без UI drill-down	Страница SKU verification с retry
Yandex media	repair-yandex-media-from-ozon - скрипт	Кнопка в UI + schedule
Группы	Расхождения Ozon/Yandex links	Авто-repair + alert на dashboard
Остатки	auto zero stock on no supplier	Настраиваемые правила per marketplace
Rate limits	BullMQ delayed jobs не видны manager	Queue timeline UI
Мульти-аккаунт	marketplace-accounts.json	UI управления аккаунтами Ozon/Yandex

9. Данные, каталог, группировка

- warehouse_products: canonical id, marketplace, offerId, grouping key
- product_links: strict identity (productId + article + partnerId + keyword)
- brand_index_items: trigram search, rebuild via API
- pm_snapshot_items: delta price push, too_many_pricemaster_changes guard
- Скрипты audit-*: catalog health, canonical mismatches, auto-pair
- Предложение: единый «Catalog Health» dashboard с score и one-click repair
- Предложение: версионирование групп Ozon/Yandex с историей в audit

10. Безопасность, производительность, эксплуатация

10.1. Безопасность

- Сессии + rate limit login (40/15min), роли admin/manager
- PM MySQL: GET_LOCK на cart, рекомендация read-only user (README)
- Rollback supplier cart требует confirm ROLLBACK_DAVIDSKLAD_SUPPLIER_CART
- Улучшить: 2FA для admin, audit log immutability, secrets rotation
- Улучшить: отдельный PM DB user только INSERT RequestDocs/Rows

10.2. Производительность

- server.js монолит - разнести по routes (уже частично: operations, settings)
- warehouse in-memory page max 5000, full load опционален

- slow endpoint tracking в /api/system/status
- PostgreSQL индексы: marketplace+status, audit createdAt
- Кеш buildWarehouseViewCached, PM snapshot indexes

10.3. DevOps

- 72 scripts/*.cjs для prod repair/audit/deploy
- smoke.test.cjs - обширные тесты API
- pm2, predeploy/postdeploy checks
- Предложение: CI pipeline, structured alerts, backup data/ automation

11. Фазируемая дорожная карта

Фаза 1 (0-6 недель) - Quick wins

Приоритет: Высокий | Трудозатраты: Низкий

- Dashboard с live-status + audit feed (polling 10s)
- Расширить nav admin: все разделы в меню
- Автокорзина: виджет ошибок skipReason + ссылка на товар
- PM status: Sended/Recieved в SupplierCartPage
- Уведомления email/Telegram при failed auto cart

Фаза 2 (6-12 недель) - Real-time + UX

Приоритет: Высокий | Трудозатраты: Средний

- SSE для operations и supplier cart
- Единый product drawer (замена legacy product.html)
- Catalog Health page из audit scripts
- Улучшенный audit с diff и фильтрами
- Mobile picking UI

Фаза 3 (3-6 мес) - Ginger/PM интеграция

Приоритет: Средний | Трудозатраты: Средний

- Двусторонняя синхронизация статусов PM/Ginger
- Idempotent cart commit с dedup по orderId
- Полный переход supplier state на PostgreSQL без JSON
- BullMQ worker split (api/worker roles)
- Финансы: автоматическая сверка PM purchase docs

Фаза 4 (6-12 мес) - Платформа

Приоритет: Средний | Трудозатраты: Высокий

- Микросервисная декомпозиция server.js
- Multi-tenant marketplace accounts
- ML scoring поставщиков для автокорзины
- Полный отказ от legacy UI
- Public API для внешних интеграций

11.1. Риски

- PriceMaster write lock contention при пиковой автокорзине
- Рассинхрон JSON fallback vs PostgreSQL
- Ozon API rate limits при mass automation
- Отсутствие документации Ginger API - нужно уточнить у команды
- Монолит server.js усложняет тестирование изменений

12. Quick wins vs долгосрочные инициативы

Quick wins (1-4 нед)	Долгосрочные (3-12 мес)
Дашборд последних действий	SSE/WebSocket платформа
Все разделы в navigation	Микросервисная архитектура
skipReason badges в автокорзине	Ginger двусторонний API
PM RequestDocs статусы в UI	ML supplier scoring
Объединить Operations + Cart UI	Полный отказ от legacy
Email alert на auto cart fail	Multi-tenant accounts
Catalog health one-click repair	Public integration API

Топ-10 рекомендаций

1. Создать главный Dashboard: статусы PM, маркетплейсов, автокорзины, очередей и лента аудита.
2. Документировать и визуализировать связь автокорзины с PriceMaster/Ginger (RequestDocs/Rows).
3. Добавить real-time (SSE) для операций и автокорзины вместо агрессивного polling.
4. Расширить навигацию modern UI - все admin-разделы доступны из меню.
5. Улучшить автокорзину: отображение skipReason, retry, PM Sended/Recieved статусы.
6. Консолидировать legacy product.html в modern Warehouse drawer.
7. Внедрить Catalog Health dashboard на базе audit-скриптов.
8. Завершить миграцию state на PostgreSQL, минимизировать JSON fallback.
9. Алерты (email/Telegram) на сбои auto cart, failed operations, PM timeout.
10. Разделить server.js на доменные модули + CI с smoke.test.cjs на каждый PR.

Документ подготовлен на основе анализа: server.js, prisma/schema.prisma, frontend/src, public/*, scripts/*, test/smoke.test.cjs.
Изменения в код не вносились.